

AIM:

To write a C program for bubble sorting.

ALGORITHM:

STEP 1: Start the program

STEP 2: PRINT Enter the number of elements :

STEP 3: READ n

STEP 4: PRINT Enter the elements :

STEP 5: Repeat STEP 1 to 3 while $i < n$;

STEP 6: PRINT $i + 1$

STEP 7: READ $a[i]$

STEP 8: SET $i := i + 1$

End of for block

STEP 9: Repeat STEP 1 to 1 while $i < n - 1$;

STEP 10: Repeat STEP 1 to 2 while $j < n - i - 1$;

STEP 11: if $a[j] > a[j + 1]$ then :

STEP 12: SET $temp := a[j]$

STEP 13: SET $a[j] := a[j + 1]$

STEP 14: SET $a[j + 1] := temp$

End of if block

STEP 15: SET $j := j + 1$

End of for block

STEP 16: SET $i := i + 1$

End f for block

STEP 17: Repeat STEP 1 to 2 while $k < n$;

STEP 18: PRINT $a[k]$

STEP 19: SET $k := k + 1$

End of for block

STEP 20: Stop the program

PROGRAM:

```
#include <stdio.h>
void main()
{
    int a[10], n;
    printf("Enter the number of elements:");
    scanf("%d", &n);
    printf("Enter the elements:");
    for (i = 0; i < n; i++)
    {
        printf("Enter element %d", i + 1);
        scanf("%d", a[i]);
    }
    int i, j, temp, k;
    for (i = 0; i < n - 1; i++)
```

```

{
    for (j = 0; j < n - i - 1; j++)
    {
        if (a[j] > a[j + 1])
        {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}
for (k = 0; k < n; k++)
    printf("%d\t", a[k]);
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
Enter the number of elements:6
Enter element 1:6
Enter element 2:4
Enter element 3:2
Enter element 4:1
Enter element 5:5
Enter element 6:3
The sorted array:1  2  3  4  5  6

```

RESULT:

Thus, the above C program for bubble sorting is successfully executed and verified successfully.

AIM:

To write a C program for Selection sorting.

ALGORITHM:

STEP 1: Start the program

STEP 2: PRINT Enter the number of elements :

STEP 3: READ n

STEP 4: Repeat

STEP 5: 1 to 3 while $i < n$;

STEP 6: PRINT $i + 1$

STEP 7: READ arr [i]

STEP 8: SET $i := i + 1$

End of for block

STEP 9: Repeat STEP 1 to 5 while $i < n - 1$;

STEP 10: SET min := i

STEP 11: Repeat STEP 1 to 2 while $j < n$;

STEP 12: if arr [j] < arr [min] then :

STEP 13: SET min := j

End of if block

STEP 14: SET $j := j + 1$

End of for block

STEP 15: SET temp := arr [min]

STEP 16: SET arr [min] := arr [i]

STEP 17: SET arr [i] := temp

STEP 18: SET $i := i + 1$

End of for block

STEP 19: Repeat STEP 1 to 2 while $i < n$;

STEP 20: PRINT arr [i]

STEP 21: SET $i := i + 1$

End of for block

STEP 22: Stop the program

PROGRAM:

```
#include <stdio.h>
void main()
{
    int arr[10], i, j, min, n, temp;
    printf("Enter the number of elements:");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("Enter element %d:", i + 1);
        scanf("%d", &arr[i]);
    }
    for (i = 0; i < n - 1; i++)
```

```

{
    min = i;
    for (j = i + 1; j < n; j++)
    {
        if (arr[j] < arr[min])
            min = j;
    }
    temp = arr[min];
    arr[min] = arr[i];
    arr[i] = temp;
}
for (i = 0; i < n; i++)
{
    printf("%d\t", arr[i]);
}
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
Enter the number of elements:5
Enter element 1:5
Enter element 2:2
Enter element 3:4
Enter element 4:1
Enter element 5:
The sorted array:5    4    3    2    1

```

RESULT:

Thus, the above C program for Selection sorting is successfully executed and verified successfully.

AIM:

To write a C program for insertion sorting.

ALGORITHM:

STEP 1: Start the program

STEP 2: PRINT Enter the number of elements :

STEP 3: READ n

STEP 4: Repeat STEP 1 to 3 while $i < n$;

STEP 5: PRINT $i + 1$

STEP 6: READ arr [i]

STEP 7: SET $i := i + 1$

End of for block

STEP 8: Repeat STEP 1 to 4 while $i < n$;

STEP 9: SET key := arr [i]

STEP 10: SET $j := i - 1$

STEP 11: Repeat STEP 1 to 2 while $j \geq 0$ and arr [j] > key

STEP 12: SET arr [j + 1] := arr [j]

STEP 13: SET $j := j - 1$

End of while block

STEP 14: SET arr [j + 1] := key

STEP 15: SET $i := i + 1$

End of for block

STEP 16: PRINT The sorted array : \n

STEP 17: Repeat STEP 1 to 2 while $i < n$;

STEP 18: PRINT arr [i]

STEP 19: SET $i := i + 1$

End of for block

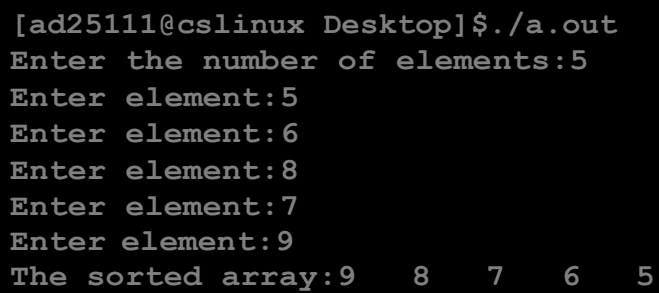
STEP 20: Stop the program

PROGRAM:

```
#include <stdio.h>
void main()
{
    int arr[10], i, key, j, n;
    printf("Enter the number of elements:");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("Enter element %d:", i + 1);
        scanf("%d", &arr[i]);
    }
    for (i = 1; i < n; i++)
    {
        key = arr[i];
        j = i - 1;
```

```
while ((j >= 0) && (arr[j] > key))
{
    arr[j + 1] = arr[j];
    --j;
}
arr[j + 1] = key;
}
printf("The sorted array:\n");
for (i = 0; i < n; i++)
{
    printf("%d\t", arr[i]);
}
}
```

SAMPLE INPUT AND OUTPUT:



```
[ad25111@cslinux Desktop]$ ./a.out
Enter the number of elements:5
Enter element:5
Enter element:6
Enter element:8
Enter element:7
Enter element:9
The sorted array:5    6    7    8    9
```

RESULT:

Thus, the above C program for insertion sorting is successfully executed and verified successfully.

AIM:

To write a C program to sort the elements of an array using merge sorting.

ALGORITHM:

Begin procedure merge(int [], int , int , int):-

STEP 1: PRINT Enter the total no. of elements :

STEP 2: READ size

STEP 3: PRINT Enter the array elements :

STEP 4: Repeat STEP 1 to 2 while $i < \text{size}$;

STEP 5: READ arr [i]

STEP 6: SET $i := i + 1$

End of for block

STEP 7: PRINT \nMerge sorted list :

STEP 8: Repeat STEP 1 to 2 while $i < \text{size}$;

STEP 9: PRINT arr [i]

STEP 10: SET $i := i + 1$

End of for block

End procedure

Begin procedure part(int arr [], int min , int max):-

STEP 11: if $\text{min} < \text{max}$ then :

STEP 12: SET $\text{mid} := (\text{min} + \text{max}) / 2$

End of if block

STEP 13: if $\text{max} - \text{min} = \text{size} / 2 - 1$ then :

STEP 14: PRINT \nHalf sorted list :

STEP 15: Repeat STEP 1 to 2 while $i \leq \text{max}$;

STEP 16: PRINT arr [i]

STEP 17: SET $i := i + 1$

End of for block

End of if block

End procedure

Begin procedure merge(int arr [], int min , int mid , int max):-

STEP 18: SET $j := \text{min}$

STEP 19: SET $m := \text{mid} + 1$

STEP 20: Repeat STEP 1 to 3 while $j \leq \text{mid}$ and $m \leq \text{max}$;

STEP 21: if $\text{arr}[j] \leq \text{arr}[m]$ then :

STEP 22: SET $\text{tmp}[i] := \text{arr}[j]$

STEP 23: SET $j := j + 1$

End of if block

STEP 24: else :

STEP 25: SET $\text{tmp}[i] := \text{arr}[m]$

STEP 26: SET $m := m + 1$

end of else block

STEP 27: SET $i := i + 1$

End of for block

STEP 28: if $j > \text{mid}$ then :

STEP 29: Repeat STEP 1 to 3 while $k \leq \text{max}$;

STEP 30: SET $\text{tmp}[i] := \text{arr}[k]$

STEP 31: SET $i := i + 1$
STEP 32: SET $k := k + 1$
 End of for block
 End of if block
STEP 33: else :
STEP 34: Repeat STEP 1 to 3 while $k \leq \text{mid}$;
STEP 35: SET $\text{tmp}[i] := \text{arr}[k]$
STEP 36: SET $i := i + 1$
STEP 37: SET $k := k + 1$
 End of for block
 End of else block
STEP 38: Repeat STEP 1 to 2 while $k \leq \text{max}$;
STEP 39: SET $\text{arr}[k] := \text{tmp}[k]$
STEP 40: SET $k := k + 1$
 End of for block
 End procedure

PROGRAM:

```

#include <stdio.h>
void merge(int[], int, int, int);
void part(int[], int, int);
int size;
void main()
{
    int i, arr[30];
    printf("Enter the total no. of elements:");
    scanf("%d", &size);
    printf("Enter the arrayelements:");
    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);
    part(arr, 0, size - 1);
    printf("\nMerge sorted list:");
    for (i = 0; i < size; i++)
        printf("%d\t", arr[i]);
}
void part(int arr[], int min, int max)
{
    int mid, i;
    if (min < max)
    {
        mid = (min + max) / 2;
        part(arr, min, mid);
        part(arr, mid + 1, max);
        merge(arr, min, mid, max);
    }
    if (max - min == (size / 2) - 1)
    {
        printf("\nHalf sorted list:");
        for (i = min; i <= max; i++)
            printf("%d\t", arr[i]);
    }
}

```



```

void merge(int arr[], int min, int mid, int max)
{
    int tmp[30];
    int i, j, k, m;
    j = min;
    m = mid + 1;
    for (i = min; j <= mid && m <= max; i++)
    {
        if (arr[j] <= arr[m])
        {
            tmp[i] = arr[j];
            j++;
        }
        else
        {
            tmp[i] = arr[m];
            m++;
        }
    }
    if (j > mid)
    {
        for (k = m; k <= max; k++)
        {
            tmp[i] = arr[k];
            i++;
        }
    }
    else
    {
        for (k = j; k <= mid; k++)
        {
            tmp[i] = arr[k];
            i++;
        }
    }
    for (k = min; k <= max; k++)
        arr[k] = tmp[k];
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
Enter the total no. of elements:5
Enter the array elements:3 9 8 4 1
Half sorted list:3 9
Half sorted list:1 4
Merge sorted list:1 3 4 8 9

```

RESULT:

Thus, the above C program for Merge sorting is successfully executed and verified successfully.

AIM:

To write a C program to sort array elements using Radix sort.

ALGORITHM:

Begin procedure getmax(int arr [], int size):-

STEP 1: SET max := arr [0]

STEP 2: Repeat STEP 1 to 2 while i < size ;

STEP 3: if arr [i] > max then :

STEP 4: SET max := arr [i]

End of if block

STEP 5: SET i := i + 1

End of for block

STEP 6: RETURN max

End procedure

Begin procedure countingsort(int arr [], int size , int div):-

STEP 8: Repeat STEP 1 to 2 while i < size ;

STEP 9: SET count [(arr [i] / div) % 10] ++ ; }

for (i = 0; i < size; i++)

STEP 10: SET i := i + 1

End of for block

STEP 11: Repeat STEP 1 to 2 while i < 10 ;

STEP 12: SET count [i] + := count [i - 1]

STEP 13: SET i := i + 1

End of for block

STEP 14: Repeat STEP 1 to 3 while i >= 0 ;

STEP 15: SET output [(count [arr [i] / div] % 10) - 1] := arr [i]

STEP 16: SET count [(arr [(i / div) % 10]) -- ;

for (i = 0; i < size; i++)

STEP 17: SET i := i - 1

End of for block

STEP 18: Repeat STEP 1 to 2 while i < size ;

STEP 19: SET arr [i] := output [i]

STEP 20: SET i := i + 1

End of for block

End procedure

Begin procedure Radixsort(int arr [], int size):-

STEP 21: SET m := getmax (arr , size)

STEP 22: Repeat STEP 1 to 1 while m / div > 0 ;

STEP 23: SET div := div * 10

End of for block

End procedure

Begin procedure main():-

STEP 24: PRINT Enter the size of array :

STEP 25: READ size

STEP 26: PRINT size

STEP 27: Repeat STEP 1 to 2 while i < size ;

STEP 28: READ arr [i]

STEP 29: SET $i := i + 1$
 End of for block
STEP 30: PRINT Before sorting : \n
STEP 31: Repeat STEP 1 to 2 while $i < \text{size}$;
STEP 32: PRINT arr [i]
STEP 33: SET $i := i + 1$
 End of for block
STEP 34: PRINT \nAfter sorting : \n
STEP 35: Repeat STEP 1 to 2 while $i < \text{size}$;
STEP 36: PRINT arr [i]
STEP 37 SET $i := i + 1$
 End of for block
 End procedure

PROGRAM:

```
#include <stdio.h>
int getmax(int arr[], int size)
{
    int max = arr[0], i;
    for (i = 1; i < size; i++)
    {
        if (arr[i] > max)
            max = arr[i];
    }
    return max;
}
void countingsort(int arr[], int size, int div)
{
    int output[size], i;
    int count[10] = {0};
    for (i = 0; i < size; i++)
        count[(arr[i] / div) % 10]++;
    for (i = 1; i < 10; i++)
        count[i] += count[i - 1];
    for (i = size - 1; i >= 0; i--)
    {
        output[(count[arr[i] / div] % 10) - 1] = arr[i];
        count[(arr[i] / div) % 10]--;
    }
    for (i = 0; i < size; i++)
        arr[i] = output[i];
}
void Radixsort(int arr[], int size)
{
    int div, m = getmax(arr, size), i;
    for (div = 1; m / div > 0; div *= 10)
        countingsort(arr, size, div);
}
void main()
{
    int i, size;
```

```

printf("Enter the size of array:");
scanf("%d", &size);
int arr[size];
printf("Enter %d integers:", size);
for (i = 0; i < size; i++)
    scanf("%d", &arr[i]);
printf("Before sorting:\n");
for (i = 0; i < size; i++)
{
    printf("%d\t", arr[i]);
}
Radixsort(arr, size);
printf("\nAfter sorting:\n");
for (i = 0; i < size; i++)
{
    printf("%d\t", arr[i]);
}
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
Enter the size of array:6
Enter 6 integers: 3 1 8 5 6 9
Before sorting:
3 1 8 5 6 9
After sorting:
1 3 5 6 8 9

```

RESULT:

Thus, the above C program for Radix sorting is successfully executed and verified successfully.

AIM:

To write a C program for Quick sorting.

ALGORITHM:

Begin procedure qsort(int arr [20], int fst , int last):-

STEP 1: PRINT Enter total No of Elements :

STEP 2: READ size

STEP 3: PRINT size

STEP 4: Repeat STEP 1 to 2 while i < size ;

STEP 5: READ arr [i]

STEP 6: SET i := i + 1

End of for block

STEP 7: PRINT \nQuick Sorted elements \n

STEP 8: Repeat STEP 1 to 2 while i < size ;

STEP 9: PRINT arr [i]

STEP 10: SET i := i + 1

End of for block

End procedure

Begin procedure qsort(int arr [20], int fst , int last):-

STEP 11: if fst < last then :

STEP 12: SET pivot := fst

STEP 13: SET i := fst

STEP 14: SET j := last

STEP 15: Repeat STEP 1 to 1 while i < j

STEP 16: Repeat STEP 1 to 1 while arr [i] <= arr [pivot] and i < last

STEP 17: SET i := i + 1

End of while block

STEP 18: Repeat STEP 1 to 1 while arr [j] > arr [pivot]

STEP 19: SET j := j - 1

End of while block

STEP 20: if i < j then :

STEP 21: SET temp := arr [i]

STEP 22: SET arr [i] := arr [j]

STEP 23: SET arr [j] := temp

End of if block

End of while block

STEP 24: SET temp := arr [pivot]

STEP 25: SET arr [pivot] := arr [j]

STEP 26: SET arr [j] := temp

End of if block

End procedure

PROGRAM:

```
#include <stdio.h>
```

```
void qsort(int arr[20], int fst, int last);
```

```
void main()
```

```

{
    int arr[30];
    int i, size;
    printf("Enter total No of Elements:");
    scanf("%d", &size);
    printf("\nEnter total %d elements:\n", size);
    for (i = 0; i < size; i++)
        scanf("%d", &arr[i]);
    qsort(arr, 0, size - 1);
    printf("\nQuick Sorted elements\n");
    for (i = 0; i < size; i++)
        printf("%d\t", arr[i]);
}
void qsort(int arr[20], int fst, int last)
{
    int i, j, pivot, temp;
    if (fst < last)
    {
        pivot = fst;
        i = fst;
        j = last;

        while (i < j)
        {
            while (arr[i] <= arr[pivot] && i < last)
                i++;
            while (arr[j] > arr[pivot])
                j--;
            if (i < j)
            {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        temp = arr[pivot];
        arr[pivot] = arr[j];
        arr[j] = temp;
        qsort(arr, fst, j - 1);
        qsort(arr, j + 1, last);
    }
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
Enter the total no. of elements:6
Enter 6 elements:3 9 1 5 8 4
Quick sorted elements:1 3 4 5 8 9

```

RESULT:

Thus, the above C program for Quick sorting is successfully executed and verified successfully.

AIM:

To write a C program to search an element in an array using linear search

ALGORITHM:

STEP 1: Start the program

STEP 2: Read number of array elements n

STEP 3: Read array elements A_i , $i = 0, 1, 2, \dots, n-1$

STEP 4: Read search value

STEP 5: Assign 0 to found

STEP 6: Check each array element against search

STEP 7: If $A_i = \text{search}$ then found = 1

STEP 8: Print "Element found" and Print position i and stop

STEP 9: If found = 0 then print "Element not found"

STEP 10: Stop the program

PROGRAM:

```
#include<stdio.h>
void main()
{
    int a[50],i, n, val, found;
    printf("Enter number of elements : ");
    scanf("%d", &n);
    printf("Enter Array Elements : \n");
    for(i=0; i<n;i++)
    {
        if(a[i]==val)
        {
            printf("Elements found at position%d",i)
            found=1;
            break;
        }
    }
    if(found==0)
    {
        printf("\nElement not found");
    }
}
```

SAMPLE INPUT AND OUTPUT:

```
[ad25111@cslinux Desktop]$ ./a.out
Enter number of elements : 7
Enter Array Elements :
23 6 12 5 0 32 10
Enter element to locate : 5
Element found at position:3

[ad25b076@cslink Desktop]$ Desktop]$ ./a.out
Enter number of elements:4
Enter Array Elements:
3 67 5 4
Enter element to locate:9
Element not found
```

RESULT:

Thus, the above C program to search an element using linear search is executed and verified successfully

AIM:

To write a C program to search an element in an array using binary search

ALGORITHM:

STEP 1: Start the program

STEP 2: Read number of array elements, say n

STEP 3: Create an array arr consisting n sorted elements

STEP 4: Get element, say key to be located

STEP 5: Assign 0 to lower and n to upper

STEP 6: While (lower < upper)

a. Determine middle element $mid = (upper + lower) / 2$

b. If $key = arr[mid]$ then

i. Print mid

ii. Stop

c. Else if $key > arr[mid]$ then

i. $lower = mid + 1$

else $upper = mid - 1$

STEP 7: Print "Element not found"

STEP 8: Stop the program

PROGRAM:

```
#include <stdio.h>
void main()
{
    int a[50], i, n, upper, lower, mid, val, found, att=0;
    printf("Enter array size : ");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        a[i] = 2 * i;
    }
    printf("\n Elements in Sorted Order \n");
    for(i=0; i<n; i++)
    {
        printf("%4d", a[i]);
    }
    printf("\n Enter element to locate : ");
    scanf("%d", &val);
    upper = n;
    lower = 0;
    found = -1;
    while (lower <= upper)
```

```

{
    mid = (upper + lower)/2;
    att++;
    if (a[mid] == val)
    {
        printf("Found at index %d in %d attempts", mid, att);
        found = 1;
        break;
    }
    else if (a[mid] > val)
        upper = mid - 1;
    else
        lower = mid + 1;
}
if (found == -1)
    printf("Element not found");
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
Enter array size : 10
Elements in Sorted Order
0 2 4 6 8 10 12 14 16 18
Enter element to locate : 16
Found at index 8 in 2 attempts

[ad23b122@cslinux Desktop]$ ./a.out
Enter array size:5
Elements in sorted order
3 5 7 9 11
Enter element to locate:2
Element not found

```

RESULT:

Thus, the above C program to search an element in an array using binary search is executed and verified successfully.

AIM:

To write a C program to create a list using array and perform operations such as display, insertion and deletion

ALGORITHM:

STEP 1:. Start the program

STEP2: Define list using an array of size n.

STEP 3: First item with subscript or position = 0

STEP 4: Display menu on list operation

STEP 5: Accept user choice

STEP 6: If choice = 1 then

STEP 7: Locate node after which insertion is to be done

STEP 8: Create an empty location in array after the node by moving the existing elements one position ahead.

STEP 9: Insert the new element at appropriate position

STEP 10: Else if choice = 2

STEP 11: Get element to be deleted.

STEP 12: Locate the position of the element replace the element with the element in the following position.

STEP 13: Repeat the step for successive elements until end of the array.

STEP 14: Else

STEP 15: Traverse the list from position or subscript 0 to n.

STEP 16: Stop the program

PROGRAM:

```
#include<stdio.h>
#include<conio.h>
#define MAX 10
void create();
void insert();
void deletion();
void search();
void display();
int a,b[20], n, p, e, f, i, pos;
void main()
{
    int ch;
    char g='y';
    do
    {
        printf("\n main Menu");
        printf("\n 1.Create \n 2.Delete \n 3.Search \n 4.Insert \n 5.Display\n 6.Exit
        printf("\n Enter your Choice");
```

```

scanf("%d", &ch);
switch(ch)
{
    case 1:
        create();
        break;
    case 2:
        deletion();
        break;
    case 3:
        search();
        break;
    case 4:
        insert();
        break;
    case 5:
        display();
        break;
    case 6:
        exit();
        break;
    default:
        printf("\n Enter the correct choice:");
}
printf("\n Do u want to continue::");
scanf("\n%c", &g);
}
while(g=='y'||g=='Y');
getch();
}
void create()
{
    printf("\n Enter the number of nodes");
    scanf("%d", &n);
    for(i=0;i<n;i++)
    {
        printf("\n Enter the Element:",i+1);
        scanf("%d", &b[i]);
    }
}
void deletion()
{
    printf("\n Enter the position u want to delete::");
    scanf("%d", &pos);
    if(pos>=n)
    {
        printf("\n Invalid Location::");
    }
    else
    {
        for(i=pos+1;i<n;i++)
        {

```

```

    b[i-1]=b[i];
}
n--;
}
printf("\n The Elements after deletion");
for(i=0;i<n;i++)
{
    printf("\t%d", b[i]);
}
}
void search()
{
    printf("\n Enter the Element to be searched:");
    scanf("%d", &e);
    for(i=0;i<n;i++)
    {
        if(b[i]==e)
        {
            printf("value is in the %d position",i);
        }
        else
        {
            printf("value is not in the %d position",i);
            continue;
        }
    }
}
void insert()
{
    printf("\n Enter the position u need to insert:");
    scanf("%d", &pos);
    if(pos>=n)
    {
        printf("\ninvalid Location:");
    }
    else
    {
        for(i=MAX-1;i>=pos-1;i--)
        {
            b[i+1]=b[i];
        }
        printf("\n Enter the element to insert:\n");
        scanf("%d",&p);
        b[pos]=p;
        n++;
    }
    printf("\n The list after insertion:\n");
    display();
}
void display()
{
    printf("\n The Elements of The list ADT are:");

```

```
for(i=0;i<n;i++)
{
    printf("\n\n%d", b[i]);
}
}
```

SAMPLE INPUT AND OUTPUT:

```
[ad25111@cslinux Desktop]$ ./a.out
Main menu
1.Create
2.Delete
3.Search
4.Insert
5.Display
6.Exit
Enter your choice:1
Enter the number of nodes:5
Enter the element1:3
Enter the element2:5
Enter the element3:2
Enter the element4:1
Enter the element5:6
Do u want to continue:y
Main menu
1.Create
2.Delete
3.Search
4.Insert
5.Display
6.Exit
Enter your choice:5
The elements of the list ADT are:
3
5
2
1
6
Do u want to continue:y
Main menu
1.Create
2.Delete
3.Search
4.Insert
5.Display
6.Exit
Enter your choice:2
Enter the position u want to delete:3
The elements after deletion:
3
5
2
```

```
6
Do u want to continue:y
Main menu
1.Create
2.Delete
3.Search
4.Insert
5.Display
6.Exit
Enter your choice:3
Enter the element to be searched:5
Value is in the 1 position
Do u want to continue:y
Main menu
1.Create
2.Delete
3.Search
4.Insert
5.Display
6.Exit
Enter your choice:4
Enter the position u need to insert:3
Enter the element to insert:9
The list after insertion:
3
5
2
9
6
Do u want to continue:n
```

RESULT:

Thus, the above C program to demonstrate array implementation of list was executed and verified successfully.

AIM:

To write a C program to create a singly linked list node and perform operations such as insertion and deletion

ALGORITHM:

STEP 1: Start the program

STEP 2: Define single linked list node as self referential structure

STEP 3: Create Head node with label = -1 and next = NULL using

STEP 4: Display menu on list operation

STEP 5 : Accept user choice

STEP 6 : If choice = 1 then

STEP 7 : Locate node after which insertion is to be done

STEP 8 : Create a new node and get data part

STEP 9 : Insert the new node at appropriate position by manipulating address

STEP 10: Else if choice = 2

STEP 11: Get node's data to be deleted.

STEP 12: Locate the node and delink the node

STEP 13: Rearrange the links

STEP 14: Else

STEP 15: Traverse the list from Head node to node which points to null

STEP 16: Stop the program

PROGRAM:

```
#include <stdio.h>
#include<stdlib.h>
struct node
{
    int label;
    struct node*next;
};
void main()
{
    int ch, fou=0;
    int k;
    struct node *h, *temp, *head, *h1;
    head = (struct node*) malloc(sizeof(struct node));
    head->label = -1;
    head->next = NULL;
    while(-1)
    {
        printf("\n\n SINGLY LINKED LIST OPERATIONS\n");
        printf("1->Add ");
        printf("2->Delete ");
        printf("3->View ");
```



```

printf("4->Exit\n");
printf("Enter your choice : ");
scanf("%d", &ch);
switch(ch)
{
    case 1:
        printf("\n Enter label after which to add : ");
        scanf("%d", &k);
        h = head;
        fou = 0;
        if(h->label == k)
            fou = 1;
        while(h->next != NULL)
        {
            if(h->label == k)
            {
                fou=1;
                break;
            }
            h=h->next;
        }
        if(h->label == k)
            fou = 1;
        if(fou != 1)
            printf("Node not found\n");
        else
        {
            temp=(struct node *)(malloc(sizeof(struct node)));
            printf("Enter label for new node: ");
            scanf("%d", &temp->label);
            temp->next = h->next;
            h->next = temp;
        }
        break;
    case 2:
        printf("Enter label of node to be deleted\n");
        scanf("%d", &k);
        fou = 0;
        h = h1 = head;
        while(h->next != NULL)
        {
            h=h->next;
            if(h->label == k)
            {
                fou = 1;
                break;
            }
        }
        if(fou == 0)
            printf("Sorry Node not found\n");
        else
        {

```

```

while (h1->next != h)
    h1 = h1->next;
    h1->next = h->next;
    free(h);
    printf("Node deleted successfully\n");
break;
case 3:
    printf("\n\n HEAD-> ");
    h=head;
    while (h->next != NULL)
    {
        h=h->next;
        printf("%d-> ",h->label);
    }
    printf("NULL");
    break;
case 4:
    exit(0);
default:
    printf("Invalid option");
}
}
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25076@cslinux Desktop]$ ./a.out
SINGLY LINKED LIST OPERATIONS
1->Add 2->Delete 3->View 4->Exit
Enter your choice : 1
Enter label after which new node is to be added :-1
Enter label for new node : 23
SINGLY LINKED LIST OPERATIONS
1->Add 2->Delete 3->View 4->Exit
Enter your choice : 1
Enter label after which new node is to be added : 23
Enter label for new node : 67
SINGLY LINKED LIST OPERATIONS
1->Add 2->Delete 3->View 4->Exit
Enter your choice : 3
HEAD-> 23-> 67-> NULL

```

RESULT:

Thus, the above C program to demonstrate list operations using linked list is executed and verified successfully

AIM:

To write a C program to implement stack operations using array

ALGORITHM:

STEP 1: Start the program

STEP 2: Define a array stack of size max = 5

STEP 3: Initialize top = -1

STEP 4: Display a menu listing stack operations

STEP 5: Accept choice

STEP 6: If choice = 1 then

STEP 7: If top < max -1

STEP 8: Increment top

STEP 9: Store element at current position of top

STEP 10: Else

STEP 11: Print Stack overflow

STEP 12: If choice = 2 then

STEP 13: If top < 0 then

STEP 14: Print Stack underflow

STEP 15: Else

STEP 16: Display current top element

STEP 17: Decrement top

STEP 18: If choice = 3 then

STEP 19: Display stack elements starting from top

STEP 20: Stop the program

PROGRAM:

```
#include <stdio.h>
#include <conio.h>
#define max 5
static int stack[max];
int top = -1;
void push(int x)
{
    stack[++top] = x;
}
int pop()
{
    return (stack[top--]);
}
void view()
{
    int i;
```

```

if (top < 0)
    printf("\n Stack Empty\n");
else
{
    printf("\n Top-->");
    for(i=top; i>=0; i--)
    {
        printf("%4d", stack[i]);
    }
    printf("\n");
}
}
void main()
{
    int ch=0, val;
    while(ch != 4)
    {
        printf("\n STACK OPERATION \n");
        printf("1.PUSH ");
        printf("2.POP ");
        printf("3.VIEW ");
        printf("4.QUIT \n");
        printf("Enter Choice : ");
        scanf("%d", &ch);
        switch(ch)
        {
            case 1:
                if(top < max-1)
                {
                    printf("\nEnter Stack element : ");
                    scanf("%d", &val);
                    push(val);
                }
                else
                {
                    printf("\n Stack Overflow \n");
                }
                break;
            case 2:
                if(top < 0)
                    printf("\n Stack Underflow \n");
                else
                {
                    val = pop();
                    printf("\n Popped element is %d\n", val);
                }
                break;
            case 3:
                view();
                break;
            case 4:
                exit(0);
        }
    }
}

```

```

        default:
            printf("\n Invalid Choice \n");
        }
    }
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
STACK OPERATION
1.PUSH 2.POP 3.VIEW 4.QUIT
Enter Choice : 1
Enter Stack element : 12
STACK OPERATION
1.PUSH 2.POP 3.VIEW 4.QUIT
Enter Choice : 1
Enter Stack element : 23
STACK OPERATION
1.PUSH 2.POP 3.VIEW 4.QUIT
Enter Choice : 1
Enter Stack element : 34
STACK OPERATION
1.PUSH 2.POP 3.VIEW 4.QUIT
Enter Choice : 1
Enter Stack element : 45
STACK OPERATION
1.PUSH 2.POP 3.VIEW 4.QUIT
Enter Choice : 3
Top--> 45 34 23 12
STACK OPERATION
1.PUSH 2.POP 3.VIEW 4.QUIT
Enter Choice : 2
Popped element is 45
STACK OPERATION
1.PUSH 2.POP 3.VIEW 4.QUIT
Enter Choice : 3
Top--> 34 23 12
STACK OPERATION
1.PUSH 2.POP 3.VIEW 4.QUIT
Enter Choice : 4

```

RESULT:

Thus, the above C program to demonstrate stack operations using array is executed and verified successfully.

AIM:

To write a C program to implement stack operations using linked list

ALGORITHM:

STEP 1: Start the program

STEP 2: Define a singly linked list node for stack

STEP 3: Create Head node

STEP 4: Display a menu listing stack operations

STEP 5: Accept choice

STEP 6: If choice = 1 then

STEP 7: Create a new node with data

STEP 8: Make new node point to first node

STEP 9: Make head node point to new node

STEP 10: If choice = 2 then

STEP 11: Make temp node point to first node

STEP 12: Make head node point to next of temp node

STEP 13: Release memory

STEP 14: If choice = 3 then

STEP 15: Display stack elements starting from head node till null

STEP 16: Stop the program

PROGRAM:

```
#include <stdio.h>
#include<stdlib.h>
struct node
{
    int label;
    struct node *next;
};
void main()
{
    int ch= 0;
    int k;
    struct node *h, *temp, *head;
    head = (struct node*) malloc(sizeof(struct node));
    head->next = NULL;
    while(1)
    {
        printf("\n Stack using Linked List \n");
        printf("1->Push ");
        printf("2->Pop ");
        printf("3->View ");
        printf("4->Exit \n");
```

```

printf("Enter your choice : ");
scanf("%d", &ch);
switch(ch)
{
case 1:
temp=(struct node *)(malloc(sizeof(struct node)));
printf("Enter label for new node : ");
scanf("%d", &temp->label);
h = head;
temp->next = h->next;
h->next = temp;
break;
case 2:
h = head->next;
head->next = h->next;
printf("Node %s deleted\n", h->label);
free(h);
break;
case 3:
printf("\n HEAD -> ");
h = head;
while(h->next != NULL)
{
h = h->next;
printf("%d -> ",h->label);
}
printf("NULL \n");
break;
case 4:
exit(0);
break;
default:
printf("Invalid option");
}
}
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
Stack using Linked List
1->Push 2->Pop 3->View 4->Exit
Enter your choice : 1
Enter label for new node : 23
New node added
Stack using Linked List
1->Push 2->Pop 3->View 4->Exit
Enter your choice : 1
Enter label for new node : 34
Stack using Linked List
1->Push 2->Pop 3->View 4->Exit

```

```
Enter your choice : 3  
HEAD -> 34 -> 23 -> NULL
```

RESULT:

Thus, the above C program to demonstrate stack operations using linked list is executed and verified successfully.

AIM:

To write a C program to implement queue operations using array

ALGORITHM:

STEP 1: Start the program
STEP 2: Define a array queue of size max = 5
STEP 3: Initialize front= rear = -1
STEP 4: Display a menu listing queue operations
STEP 5: Accept choice
STEP 6: If choice = 1 then
STEP 7: If rear < max -1
STEP 8: Increment rear
STEP 9: Store element at current position of rear
STEP 10: Else
STEP 11: Print Queue full
STEP 12: If choice = 2 then
STEP 13: If front=-1 then
STEP 14: Print Queue empty
STEP 15: Else
STEP 16: Display current front element
STEP 17: Increment front
STEP 18: If choice = 3 then
STEP 19: Display queue elements starting from front to rear
STEP 20: Stop the program

PROGRAM:

```
#include <stdio.h>
#include <conio.h>
#define max 5
static int queue[max];
int front = -1;
int rear = -1;
void insert(int x)
{
    queue[++rear] = x;
    if (front == -1)
        front = 0;
}
int remove1()
{
    int val;
    val = queue[front];
```

```

if (front==rear && rear==max-1)
    front = rear = -1;
else
    front ++;
    return(val);
}
void main()
{
    int ch=0, val;
    while(ch != 3)
    {
        printf("\nQUEUE OPERATION \n");
        printf("1.ENQUEUE ");
        printf("2.DEQUEUE");
        printf("3.QUIT \n");
        printf("Enter Choice : ");
        scanf("%d", &ch);
        switch(ch)
        {
            case 1:
                if(rear < max-1)
                {
                    printf("\nEnter element to be inserted: ");
                    scanf("%d", &val);
                    insert(val);
                }
                else
                {
                    printf("\nQueue full \n");
                }
                break;
            case 2:
                if(front==-1)
                    printf("\nQueue empty \n");
                else
                {
                    val = remove();
                    printf("\n element deleted is %d\n", val);
                }
                break;
            case 3:
                exit(0);
            default:
                printf("\n Invalid Choice \n");
        }
    }
}

```

SAMPLE INPUT AND OUTPUT:

```
[ad25111@cslinux Desktop]$ ./a.out
QUEUE OPERATION
1.INSERT 2.DELETE 3.VIEW 4.QUIT
Enter Choice : 1
Enter element to be inserted : 12
QUEUE OPERATION
1.INSERT 2.DELETE 3.VIEW 4.QUIT
Enter Choice : 1
Enter element to be inserted : 23
QUEUE OPERATION
1.INSERT 2.DELETE 3.VIEW 4.QUIT
Enter Choice : 1
Enter element to be inserted : 34
QUEUE OPERATION
1.INSERT 2.DELETE 3.VIEW 4.QUIT
Enter Choice : 1
Enter element to be inserted : 45
QUEUE OPERATION
1.INSERT 2.DELETE 3.VIEW 4.QUIT
Enter Choice : 1
Enter element to be inserted : 56
QUEUE OPERATION
1.INSERT 2.DELETE 3.VIEW 4.QUIT
Enter Choice : 1
Queue Full
QUEUE OPERATION
1.INSERT 2.DELETE 3.VIEW 4.QUIT
Enter Choice : 3
Front--> 12 23 34 45 56 <--Rear
```

RESULT:

Thus, the above C program to demonstrate queue operations using array is executed and verified successfully.

AIM:

To write a C program to implement queue operations using linked list

ALGORITHM:

STEP 1: Start the program

STEP 2: Define a singly linked list node for queue

STEP 3: Create Head node

STEP 4: Display a menu listing queue operations

STEP 5: Accept choice

STEP 6: If choice = 1 then

STEP 7: Create a new node with data

STEP 8: Make new node point to first node

STEP 9: Make head node point to new node

STEP 10: If choice = 2 then

STEP 11: Make temp node point to first node

STEP 12: Make head node point to next of temp node

STEP 13: Release memory

STEP 14: If choice = 3 then

STEP 15: Display queue elements starting from head node till null

STEP 16: Stop the program

PROGRAM:

```
#include <stdio.h>
#include<stdlib.h>
struct node
{
    int label;
    struct node *next;
};
void main()
{
    int ch = 0;
    int k;
    struct node *h, *temp, *head;
    head = (struct node*) malloc(sizeof(struct node));
    head->next = NULL;
    while(1)
    {
        printf("\n Stack using Linked List \n");
        printf("1->Push ");
        printf("2->Pop ");
        printf("3->Exit\n ");
```

```

printf("Enter your choice : ");
scanf("%d", &ch);
switch(ch)
{
case 1:
temp=(struct node *)(malloc(sizeof(struct node)));
printf("Enter label for new node : ");
scanf("%d", &temp->label);
h = head;
while(h->next!=NULL)
    h= h->next;
h->next = temp;
temp->next=NULL;
break;
case 2:
h = head->next;
head->next = h->next;
printf("Node %d deleted\n", h->label);
free(h);
break;
case 3:
exit(0);
break;
default:
printf("Invalid option");
}
}
}

```

SAMPLE INPUT AND OUTPUT:

```

[ad25111@cslinux Desktop]$ ./a.out
Queue using Linked List
1->Insert 2->Delete 3->Exit
Enter your choice : 1
Enter label for new node : 23
Queue using Linked List
1->Insert 2->Delete 3->Exit
Enter your choice : 1
Enter label for new node : 34
Queue using Linked List
1->Insert 2->Delete 3->Exit
Enter your choice:1
Enter label for new node:45
Queue using Linked List
1->Insert 2->Delete 3->Exit
Enter your choice:2
Node deleted:23

```

```
Queue using Linked List  
1->Insert 2->Delete 3->Exit  
Enter your choice : 3  
HEAD -> 34 -> 45 -> NULL
```

RESULT:

Thus, the above C program to demonstrate queue operations using linked list is executed and verified successfully.